07-2-rotation

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 7 deliverable: rotation mechanism, interval, emergency procedure, and target rotation time per credential.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/06-secrets-rotation.md`

Section 6 — secrets delivery, rotation, and key custody

Threat **T3**. Artefacts: `k8s/externalsecret.yaml`, `terraform/kms.tf`, `iam/pod-identity-policy.json`.

Delivery path — source of truth to running process

```
AWS Secrets Manager  prod/accounts-api/db        ← source of truth
   │  encrypted with CMK alias/accounts-api-secrets (terraform/kms.tf)
   │  key policy: only the pod role may Decrypt, only kms:ViaService=secretsmanager
   ▼
Secrets Manager VPC endpoint (vpce-0a1b…)         ← IAM condition pins the credential
   │                                                 to our network (pod-identity-policy)
   ▼
IRSA: role/accounts-api-pod, trust scoped to ns/accounts:sa/accounts-api
   │  policy: GetSecretValue on ONE secret ARN, AWSCURRENT stage only
   ▼
Secrets Store CSI driver
   │  mounts as tmpfs at /var/run/secrets/db/credential.json, mode 0400, uid 10001
   ▼
accounts-api process — reads the file, watches it, re-reads on change
```

Nothing in this repository contains the credential, and no committed manifest contains a reference that yields it without the pod's IAM identity. What is committed is the *name* of the secret, which is not a secret.

Three properties I chose the CSI driver over External Secrets Operator to get:

- the credential **never becomes a Kubernetes Secret**, so it is not in etcd and an etcd backup does not contain it;
- `get secrets` in `ns/accounts` does not yield it — which is what let the remediated RBAC in section 4a drop secret access entirely rather than merely narrow it;
- rotation reaches a running pod **without a restart**, which is the whole reason the 5-minute emergency target below is achievable.

The application's master password is a fourth thing again: it is RDS-managed (`manage_master_user_password`) and the application never sees it. The app authenticates with an IAM auth token where it can, and with the scoped application credential above where it cannot.

Rotation

credential	mechanism	scheduled interval	emergency target	what bounds the emergency time
`prod/accounts-api/db` (app DB user)	Secrets Manager rotation Lambda, alternating-user strategy (`accounts_api_a` / `accounts_api_b`)	**30 days**	**≤ 5 min**	Lambda run ~40s + CSI `rotationPollInterval` 60s + app file-watch reload ~1s. No pod restart, no connection drain: the app opens new connections with the new credential and lets the old pool drain.
RDS master (`rdsadmin`)	RDS-managed rotation	**90 days**	**≤ 15 min**	Not used by the application, so rotating it cannot cause an outage. Longer target only because a master rotation is serialised behind any in-flight maintenance.
`prod/accounts-api/kyc` (third-party API key)	manual — the provider has no rotation API	**180 days**	**≤ 4 h**	**Bounded by the vendor, not by us.** Their support desk issues a new key; we cannot beat their SLA. This is the weakest link and I am naming it rather than averaging it into a headline number. Compensating: the key is scoped to a read-only KYC endpoint, and the egress pin means a stolen key is only usable from an attacker's own network against the vendor, not against us.
pod IRSA credentials	STS, automatic	1 h token lifetime	**immediate**	Attach a `Deny *` inline policy to `role/accounts-api-pod` ; effective on the next API call, ≤ 1 h for already-minted sessions. The residual hour is real and I would rather state it than claim revocation is instant.
GitHub Actions → AWS	OIDC, no standing credential	n/a	**immediate**	Nothing to rotate. Removing the trust policy condition kills it. This is the argument for OIDC in one row.
Cosign signing key	none exists (keyless)	n/a	n/a	Same argument. The certificate lives ~10 minutes.

Target time to rotate every credential this service holds: 15 minutes, with the honest exception of the KYC vendor key at 4 hours, which is a vendor dependency rather than an engineering one.

The alternating-user strategy, and why it matters at 03:00

Single-user rotation changes the password on the one database user. There is a window — however short — where connections opened with the old password are rejected while the new one propagates, i.e. rotation causes an outage, i.e. **nobody runs it during an incident, which is exactly when it is needed.** The alternating-user strategy keeps two database users and flips between them: the old credential stays valid until the next rotation, so propagation is not a cliff. That is what makes an emergency rotation a routine action rather than a change request.

Emergency rotation, in the order you would actually do it

```
# 1. rotate now (~40s). Does not wait for the schedule.
aws secretsmanager rotate-secret --secret-id prod/accounts-api/db \
  --client-request-token "inc-$(date +%s)"

# 2. confirm the mounted file changed in every replica (~60s later)
kubectl -n accounts get pods -l app=accounts-api -o name | while read -r p; do
  kubectl -n accounts exec "$p" -- \
    stat -c '%n %Y' /var/run/secrets/db/credential.json
done

# 3. revoke, do not merely supersede. Rotation leaves AWSPREVIOUS usable —
#    which is the point in normal operation and wrong during an incident.
psql -h accounts-prod... -c "ALTER ROLE accounts_api_b NOLOGIN;"

# 4. prove the old credential is dead, from the database's point of view
psql -c "SELECT usename, count(*) FROM pg_stat_activity GROUP BY 1;"
```

Step 3 is the one people forget. Rotation without revocation means the attacker's copy keeps working for another 30 days.

If rotation itself is what fails

The rotation Lambda is a single point of failure and it lives in the same account as the thing it protects. The manual fallback — `psql` from a break-glass bastion with the RDS-managed master credential, then `update-secret-version-stage` — is documented in the on-call runbook and rehearsed quarterly. Untested rotation is not rotation, and this is the specific piece I would insist on rehearsing rather than reasoning about.

Encryption — one line each, and what it actually addresses

At rest: RDS storage, snapshots, Performance Insights, ECR layers, the CloudTrail archive and the Secrets Manager envelope are all encrypted with **customer-managed KMS keys** (`terraform/kms.tf`), single-region for RBI localisation, annual automatic rotation, with key policies that scope `Decrypt` to one principal via one service.

What that actually defends against, honestly: disk-level and snapshot-level exposure — a snapshot shared or copied to another account, a decommissioned volume, an AWS operator with storage access. Customer-managed keys buy three more things a bank is asked about: revocation (disabling the key kills access to every copy at once, including backups an attacker has taken), an independent audit trail (every `Decrypt` is a CloudTrail event attributable to a principal), and custody that is ours rather than AWS's.

What it does not defend against — the important half: every threat in the top five. T1, T2, T3 and T5 all involve an attacker holding a *legitimate identity*, and encryption at rest is transparent to a legitimate identity. The compromised pod reads plaintext rows; the public database in section 4b would have served plaintext over TLS to the internet whether or not the disk was encrypted. So the honest statement is: **at-rest encryption is a regulatory and disk-disposal control, not a defence against anything in my threat model.** It is in this submission because a bank's examiner requires it and because it is nearly free — not because I think it stops the attacker I am worried about.

In transit: TLS 1.2+ everywhere, enforced rather than configured — `rds.force_ssl = 1` in the parameter group so the database refuses plaintext, and Istio mTLS `STRICT` inside the mesh. That one does address a real threat: a co-tenant on the shared cluster who can position on the pod network.